

Assignment 15

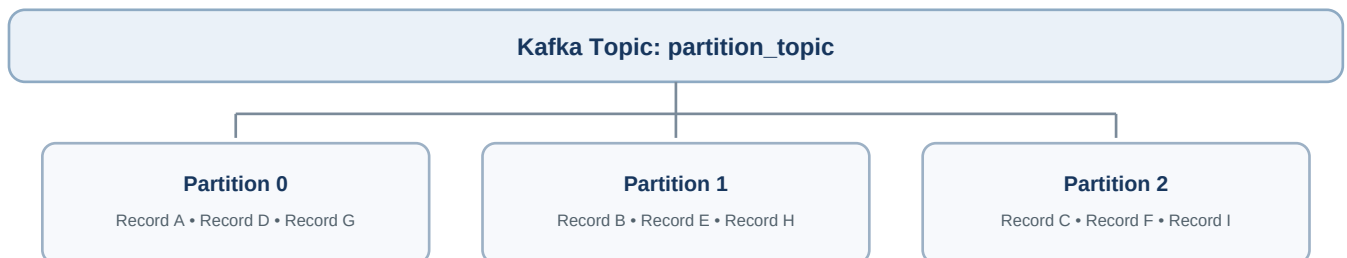
Apache Kafka Partitioning, Replication and PySpark Producer

1. Corrected Assignment Questions

- What is the replication factor in Kafka, and how does it work?
- What are Kafka partitions, how is incoming data distributed among them, and why is partitioning needed?
- How can PySpark act as a Kafka producer and send data to a Kafka topic?
- What does round-robin partitioning mean, and when is it used?
- How does Kafka use a key and a hashing algorithm to select a partition? Explain with an example.
- How can records be sent to specific Kafka partitions based on conditions using PySpark?

2. What Is a Kafka Partition?

A Kafka topic is a logical category in which messages are stored. A topic can be divided into multiple **partitions**. Each partition is an ordered, append-only sequence of records. Kafka uses partitions to distribute data and enable consumers to process different parts of a topic in parallel.



Illustrative example: a topic split into three partitions. Actual record placement depends on the producer's partitioning strategy.

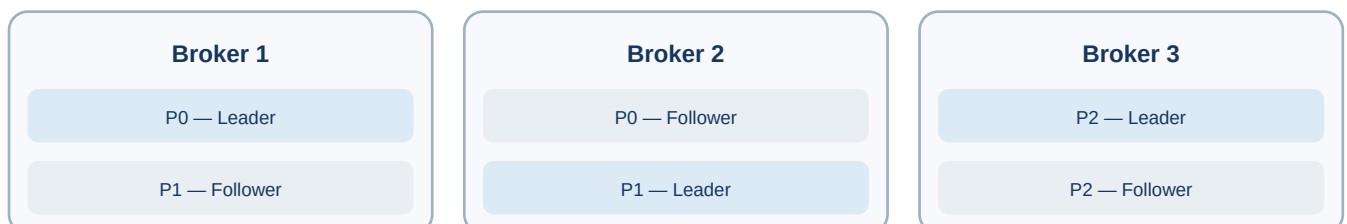
Why do we need partitions?

- **Scalability:** Data can be spread across multiple partitions and brokers.
- **Parallel processing:** Different consumers in the same consumer group can process different partitions.
- **Throughput:** Producers and consumers can work with multiple partitions concurrently.
- **Ordering:** Records are ordered within an individual partition. Kafka does not guarantee one total order across all partitions.

Adding partitions does not automatically make every consumer faster. The number of active consumers in a consumer group and the workload also matter.

3. What Is the Replication Factor?

The **replication factor** specifies how many copies of each partition Kafka maintains across brokers. For example, with three partitions and replication factor two, Kafka maintains two replicas of each partition: one leader replica and one follower replica.



Conceptual illustration only. Kafka places replicas across brokers according to its replica assignment; this layout is not a required placement.

- **Leader replica:** Handles reads and writes for that partition.
- **Follower replica:** Replicates data from the leader and may take over if an eligible leader fails.

- **Replication factor = 1:** One copy of each partition; no replica-based redundancy.
- **Replication factor = 3:** Three copies of each partition, assuming enough brokers are available.

Important: A replication factor of three requires at least three brokers to place all replicas on separate brokers. Replication improves fault tolerance, but it is not a substitute for backups.

Partition count vs replication factor

Setting	Controls	Example
Partitions	Number of logical data streams in a topic	3 partitions
Replication factor	Number of copies of each partition	2 copies per partition

With three partitions and replication factor two, the topic has six partition replicas in total, including the three leaders.

4. How Does Kafka Choose a Partition?

When a producer sends a record, the partition can be selected in three main ways:

- **Explicit partition:** The producer specifies a partition number, such as 0 or 1. Kafka sends the record there, provided that partition exists.
- **Key-based partitioning:** The producer supplies a key. The partitioner uses the key to select a partition. Records with the same key generally go to the same partition when the partition count and strategy remain unchanged.
- **No key and no explicit partition:** The producer's default partitioner chooses a partition. Exact behavior depends on the producer/version and configuration.

Is the default always round-robin?

The class notes describe the no-key case as **round-robin**. As an introductory model, records are sent to partitions in rotation.

Record	Partition
Record 1	0
Record 2	1
Record 3	2
Record 4	0
Record 5	1
Record 6	2

However, this is not universal for every Kafka producer. Modern producers can use a **sticky partitioning strategy** for records without keys, batching records into one partition for a time before switching. Distribution depends on producer implementation and configuration.

5. How Does Key-Based Hashing Work?

Suppose a topic has four partitions numbered 0, 1, 2 and 3. A simplified explanation is:

partition = positiveHash(key) % number_of_partitions

A hash function converts the key into a number. The modulo operation (%) maps that number into the valid partition range.

Key	Illustrative hash	Hash % 4	Partition
customer_101	125	1	1
customer_202	246	2	2
customer_303	311	3	3
customer_101	125	1	1

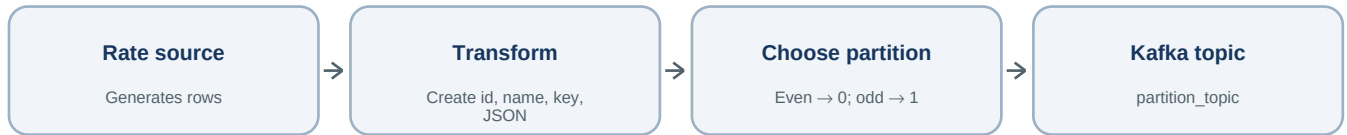
The hash values above are made-up values to demonstrate the calculation. Kafka's standard Java producer partitioner uses a specific hash implementation (commonly Murmur2) and maps the result to the available partitions. Do not expect the illustrative

values to match Kafka's actual hash values.

If the number of partitions changes, a key may map to a different partition. A key helps preserve ordering for records with that key because they are routed to the same partition; it does not guarantee ordering across the whole topic.

6. Using PySpark as a Kafka Producer

PySpark can produce records to Kafka using Structured Streaming with the Kafka sink. The class example uses Spark's built-in rate source to generate sample records, constructs a key and JSON value, chooses a partition, and writes to Kafka.



Producer flow: Spark rate source → transformations → partition selection → Kafka topic.

7. Complete PySpark Producer Code

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *

# 1. Create SparkSession
spark = (
    SparkSession.builder
        .appName("KafkaPartitionProducer")
        .config(
            "spark.jars.packages",
            "org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.6"
        )
        .getOrCreate()
)

# 2. Generate streaming data
df = (
    spark.readStream
        .format("rate")
        .option("rowsPerSecond", 10)
        .load()
)

# 3. Create id from the generated value
df = df.select(
    "value",
    col("value").cast("long").alias("id")
)

# 4. Create a name column
kafka_df = df.select(
    "id",
    concat(
        lit("name_"),
        col("value").cast("string")
    ).alias("name")
)

# 5. Create Kafka key, JSON value, and partition
kafka_df = (
    kafka_df
        .withColumn("key", col("id").cast("string"))
        .withColumn(
            "value",
            to_json(struct("id", "name"))
        )
        .withColumn(
            "partition",
            expr("""
                CASE
                    WHEN id % 2 = 0 THEN 0
                    ELSE 1
                END
            """)
        )
        .select("key", "value", "partition")
)

# 6. Write records to Kafka
query = (
    kafka_df.writeStream
        .format("kafka")
        .option("kafka.bootstrap.servers", "localhost:9092")
        .option("topic", "partition_topic")
        .option(
            "checkpointLocation",
            "file:///C:/tmp/kafka_partition_topic"
        )
        .outputMode("append")
        .trigger(processingTime="5 seconds")
        .start()
)

query.awaitTermination()
```

8. How Conditional Partitioning Works

```
.withColumn(
    "partition",
    expr("""
        CASE
            WHEN id % 2 = 0 THEN 0
            ELSE 1
        END
    """)
)
```

The condition checks whether id is divisible by two:

- If `id % 2 == 0`, the ID is even, so the record goes to partition 0.
- Otherwise, the ID is odd, so the record goes to partition 1.

id	name	key	partition
0	name_0	"0"	0
1	name_1	"1"	1
2	name_2	"2"	0
3	name_3	"3"	1
4	name_4	"4"	0
5	name_5	"5"	1

The Kafka sink recognizes the `partition` column as the destination partition. Because the partition is explicitly supplied, the condition—not key hashing—determines where each record is sent. The key is still included in the Kafka record.

This example assumes `partition_topic` has at least two partitions, numbered 0 and 1. If the topic has only one partition, selecting partition 1 will fail.

9. What Does `trigger(processingTime='5 seconds')` Mean?

```
.trigger(processingTime="5 seconds")
```

This configures Spark Structured Streaming to process available data in micro-batches at approximately five-second intervals. It does not mean that each individual record is delayed by exactly five seconds. Actual processing time can vary with workload, scheduling and Kafka availability.

- The rate source generates rows at the configured rate of 10 rows per second.
- Spark processes available rows in micro-batches.
- The Kafka sink publishes records from each micro-batch.
- Spark attempts to start the next micro-batch according to the five-second trigger interval.

10. Key vs Partition — Which Takes Priority?

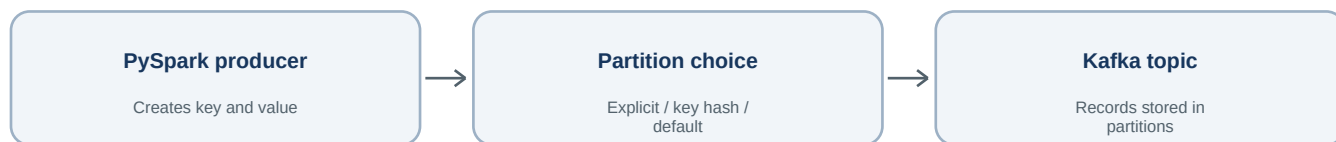
Producer configuration	How the partition is selected
Explicit partition provided	The specified partition is used.
No partition, key provided	The producer's partitioner uses the key to select a partition.
No partition, no key	The producer's default partitioning strategy is used.

In the class code, both `key` and `partition` are provided. The explicit partition determines where the record is sent. The key remains part of the Kafka record but does not override the `partition` column.

11. Important Points to Remember

- A **partition** divides a topic into separate ordered logs.
- A **replication factor** controls how many copies of each partition Kafka maintains.
- A **key** can route related records to the same partition.
- **Round-robin** is a simple model for distributing keyless records, but the actual default can depend on producer version and configuration.
- A **partition column** in the Spark Kafka sink lets you explicitly choose the destination partition.
- `trigger(processingTime='5 seconds')` controls the micro-batch interval; it is not an exact per-record delay.
- The topic must exist, and the selected partition number must be valid.

12. Final Summary Diagram



Key takeaway: partitions provide scalability and parallelism; replication provides redundancy; keys support consistent routing; and an explicit partition column lets PySpark direct records to a chosen partition.